

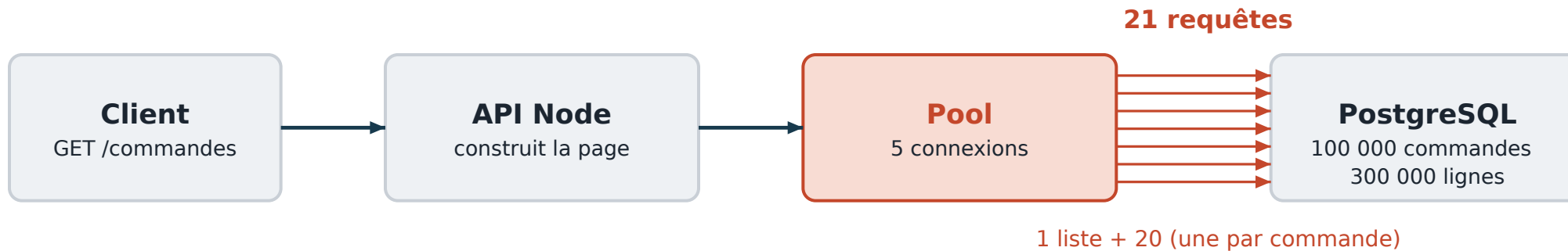
Pourquoi une page de 20 commandes ralentit sous charge, et comment la corriger

PROBLÉMATIQUE

D'où vient ce coût, comment le supprimer sans changer le résultat, et à quel prix ?

Une page = 21 requêtes SQL

GET /commandes : une page de 20 commandes avec leurs lignes



≈ **247**

requêtes/s : le débit plafonne
dès 5 clients

98 ms

latence p95 à 20 clients

15

requêtes en attente d'une
connexion

0

erreur : saturé, pas planté

À RETENIR

Une seule page déclenche 21 allers-retours vers la base : le service est saturé à 247 pages/s, sans aucune erreur.

Le coût est dans les allers-retours

2 / 7

OÙ EST LE COÛT ? TEMPS D'UNE REQUÊTE DE LIGNES

Vu par PostgreSQL

 0,018 ms

Vu par l'API (trajet inclus)

 ≈ 0,68 ms

× 38 : le temps perdu est dans les trajets, pas dans la requête

AVANT : 21 requêtes

1 requête : la liste des 20 commandes

+ 20 requêtes : une par commande pour ses lignes

APRÈS : 2 requêtes

1 requête : la liste des 20 commandes

+ 1 requête : les lignes de TOUTES les commandes d'un coup

LE RAISONNEMENT

21 trajets
≈ 14 ms par page

2 trajets
≈ 2 ms par page

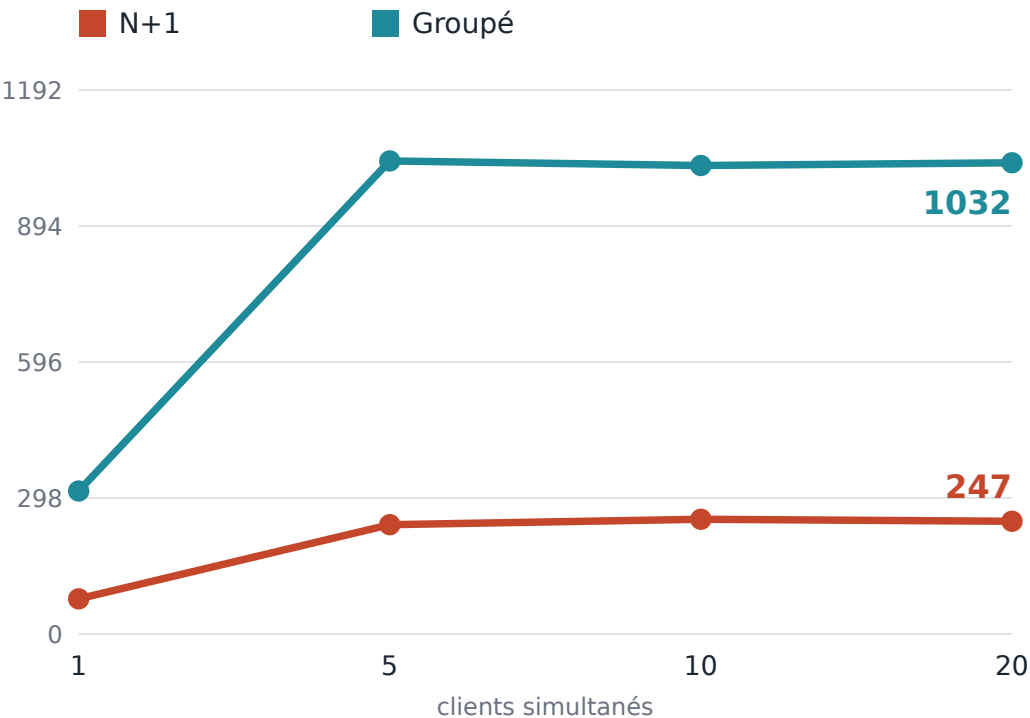
Un seul changement : la cause est donc certaine.

À RETENIR

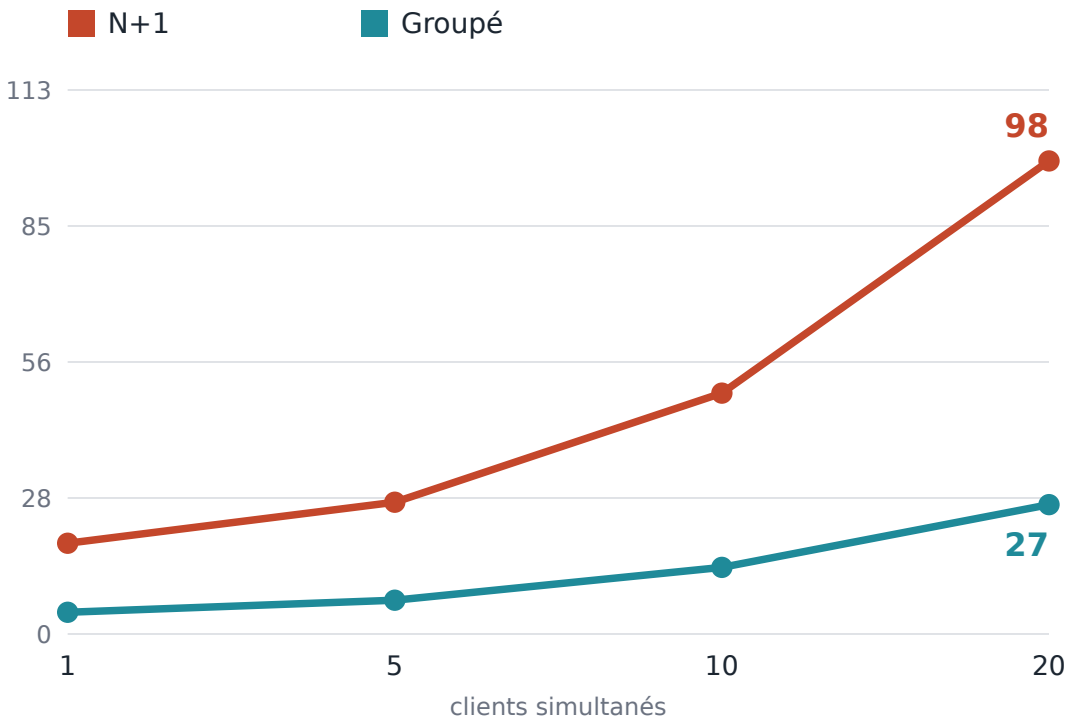
Ce n'est pas la requête qui est lente, ce sont les trajets. Je ne change qu'une chose : charger toutes les lignes en une fois.

Débit x4,2 et p95 –73 %

Pages servies par seconde (débit)



Latence p95 (ms) : les cas les plus lents



À 20 clients	Avant (N+1)	Après (groupé)	Gain
Débit (pages/s)	247	1 032	x4,2
p95 (ms)	98	27	-73 %
Erreurs	0	0	=

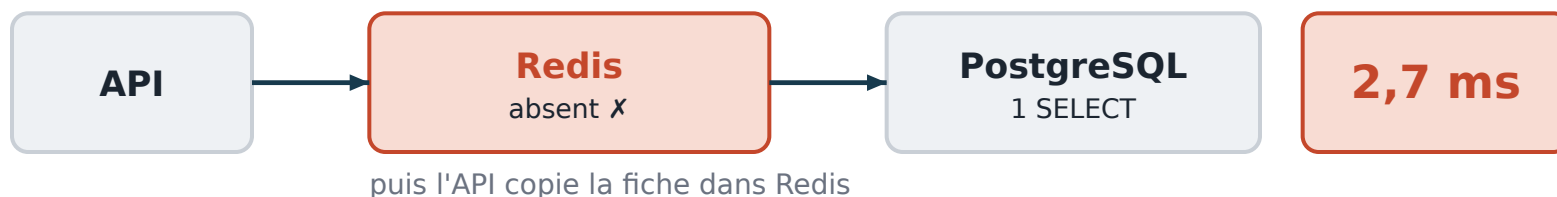
À RETENIR

Mêmes conditions, plusieurs essais : le N+1 plafonne, le groupé monte. Zéro erreur.

Lire sans interroger la base

LECTURE D'UNE FICHE PRODUIT (LUE SOUVENT, MODIFIÉE RAREMENT)

1er accès : MISS



Accès suivants : HIT



DANS LE TEMPS (TTL = 60 s)



la lecture suivante relit PostgreSQL et recopie la nouvelle valeur

Si la fiche est modifiée par l'API, la copie est supprimée tout de suite.

POURQUOI ÇA AIDE

- Lecture $\approx 2,7\times$ plus rapide
- La base est moins sollicitée
- Absorbe les pics de lecture
- Si Redis tombe : on relit la base

LIMITES

- Donnée qui change souvent : beaucoup de miss, donc moins utile
- La copie peut être périmée
- Jamais pour l'achat ou la commande

À RETENIR

Le cache garde une copie des données très lues : un « hit » ne fait aucune requête SQL. Utile si on lit souvent et modifie rarement.

Même résultat. Le cache, lui, a un prix

✓ LE RÉSULTAT EST IDENTIQUE

- Mêmes commandes, mêmes lignes qu'avant
- Mêmes droits d'accès (pas de jeton : refusé)

GAIN DU CACHE (LECTURE D'UNE FICHE PRODUIT)

2,7 ms → 0,99 ms

LE CACHE PEUT MENTIR



Prix modifié directement dans la base : le cache ne le sait pas

✓ Passer par l'API corrige la copie

Après une modification par l'API, le cache est vidé et affiche la bonne valeur. Le délai de 60 s limite la durée d'une erreur.

CE QU'ON RETIENT

1. Corriger la cause d'abord : le chargement groupé gagne du temps sans rien sacrifier.
2. Le cache va plus loin (il supprime même la requête), mais la donnée peut être périmée : c'est son prix.
3. À utiliser pour ce qu'on lit souvent et modifie rarement (fiche produit).
4. Jamais pour l'achat ou la commande : il faut la valeur exacte.

À RETENIR

Plus rapide ne suffit pas : le résultat est identique. Le cache gagne du temps, mais au prix de la fraîcheur.

Je retiens le chargement groupé

Trois façons de lever le plafond de 247 pages/s à 20 clients. Laquelle supprime vraiment les 21 trajets ?

OPTION 1

Agrandir le pool

5 → 20 connexions

≈ 287

pages/s (2 essais)



avant : 247

REJETÉ

Supprime l'attente, mais les 21 trajets par page restent.

OPTION 2

Ajouter PgBouncer

40 → 5 connexions

261 → 94

transactions/s (autre mesure)

autre unité : pas de barre comparable

REJETÉ

Borne les connexions, n'accélère rien : le débit baisse.

OPTION 3

Chargement groupé

21 → 2 requêtes

1 032

pages/s, 0 erreur



avant : 247

RETENU

Supprime les trajets : ×4,2, résultat identique.

RISQUE RESTANT

Cache de la fiche produit : copie périmée jusqu'à 60 s.
Jamais sur l'achat.

PORTÉE

Cette machine, ce jeu de données, 1 à 20 clients. Pas testé en production.

Reproductibilité et conséquence pour l'API

7 / 7

« COMMENT REPRODUIRE SUR UN AUTRE POSTE ? »

- Même base de données (mêmes scripts)
- Mêmes versions logicielles
- Même charge : 1, 5, 10 et 20 clients
- Plusieurs essais alternés avant / après
- On compare l'ordre de grandeur, pas la valeur exacte

« QU'EST-CE QUE ÇA CHANGE POUR L'API ? »

- Le contrat ne change pas : mêmes réponses, mêmes droits
- Moins de connexions occupées
- Avec le cache : fraîcheur limitée à 60 s
- Si le cache tombe : on relit la base
- Jamais de cache sur l'achat

Phrase finale : le coût venait des allers-retours ; mesuré ici, pas encore en production.